Generate a unique ID C++

Asked 1 year, 8 months ago Modified 1 year, 8 months ago Viewed 1k times

0

I want to generate a unique identifier "ident" for my complex structure, how can i do that?

in my header, the complexe structure is:

```
struct Complexe {
    float x;
    float y;
    static unsigned int ident;
};

void Init(Complexe&);

etc...
```

and in the cpp file, i need to attribute ident a unique int

```
Init(Complexe&z){
    z.ident = 0;
    z.y = 0;
    z.x = 0;
};
```

c++ struct identifier

Share Improve this question Follow

edited Apr 5, 2021 at 11:42
KamilCuk
108k 7 48 91

asked Apr 5, 2021 at 10:19
Random
41 6

1 Use `0` (or `1`) for the first ident, `1` for the next and so on. If your program doesn't run millions of times per second you'll be good for a few thousand years. – pmg Apr 5, 2021 at 10:23

Unrelated: use `double` for floating-point variables (unless you're aiming for severely restricted environments); forget `float` exists. – pmg Apr 5, 2021 at 10:26

Unrelated: C has had complex numbers as part of the language for more than 20 years. Think of `#include <complex.h>` ? – pmg Apr 5, 2021 at 10:29

Thanks, i'm using float because it's requested in the school work and they want us to manipulate complexe using struct for now and not the #include <complex.h> – Random Apr 5, 2021 at 10:44

1 Study for inspiration the source code of RefPerSys. It has objects with a unique id. Or email me (in English or in French) to `basile@starynkevitch.net` – Basile Starynkevitch Apr 5, 2021 at 10:57

Join Stack Overflow to find the best answer to your technical question, help others answer theirs. Sign up

4 Answers

Sorted by:

Highest score (default)

1

May I recommend you `std::hash` ?

```
std::size_t ident = std::hash<Complex>()(complexVar);
```

Writing it from memory but it should return you unique value (with very small chance of it being not) for each `Complex` type object.

Share Improve this answer Follow

answered Apr 5, 2021 at 10:38
trofchik
87 1 7

1

Consider UUID, specifically uuid_generate on GNU/Linux, or (it seems) UuidCreate on Windows.

Share Improve this answer Follow

answered Apr 5, 2021 at 10:55
Steve
11 4

1

Generating a unique id is easy even if you want to write your own algorithm. Though your algorithm will need some thoughts if the environment you are working in is multi-threaded. In this case you will need to write a thread safe code. for example below code will generate a unique id and is also thread safe:

```
class Utility {
public :
    static int getUniqueId();
};

int Utility::getUniqueId() {
    static std::atomic<std::uint32_t> uid { 0 };
    return ++uid;
}
```

Share Improve this answer Follow

answered Apr 5, 2021 at 11:18
Abhishek Chandel
1,234 2 13 19

1 I'd suggest using an explicit `fetch_add(1, std::memory_order_relaxed)` , since you don't actually need or want synchronization, just atomicity. – Hasturkun Apr 5, 2021 at 12:17

@Hasturkun , thanks for the food for thought. I will check it. – Abhishek Chandel Apr 6, 2021 at 0:18

Join Stack Overflow to find the best answer to your technical question, help others answer theirs.

Sign up

1

One simple way is to just make a free-list to help you reuse IDs. So for instance you start at ID 0 and whenever you create a structure, you first check the free-list for any released IDs. It will be empty the first time through so you increment the ID counter to 1 and use that for your new structure. When you destroy a structure it's ID goes back on the free-list (which can be implemented as a stack), so that the next time you create a structure that ID will be reused. If the free-list runs out of IDs you just start incrementing the ID counter from where you left off....wash, rinse and repeat. The nice thing about this method is that you will never wrap integer range and accidentally use an already in use ID if your program ends up running a long time. The main down side is the extra storage for the stack, but the stack can always grow as you need it.

Share Improve this answer Follow edited Apr 5, 2021 at 16:38 answered Apr 5, 2021 at 11:36

user13533058

Join Stack Overflow to find the best answer to your technical question, help others answer theirs.

Sign up